

Micro-Proyecto No. 2

Manipulating images with Parallel programming

Repositorio: https://github.com/japeto/netpbm_filters

Descripción general

A continuación se presentan los lineamientos para el desarrollo de múltiples diseños multihilo (*memoria compartida*) aplicadas al procesamiento de imágenes. La siguiente lista de desafíos fue diseñada para poner en práctica los conocimientos adquiridos en el curso de programación paralela.

Objetivos

Al completar este micro-proyecto, los estudiantes aprenderán a:

1. Identificar varios aspectos relacionados con la programación paralela
2. Analizar un problema y generar una solución usando programación paralela
3. Utilizar librerías/bibliotecas para paralelización de memoria compartida y distribuida
4. Determinar cuáles ventajas y desventajas tiene una solución paralela

Pregunta de Interés

¿Cómo influye la programación paralela, utilizando OpenMP ó Pthreads y MPI (En un entorno simulado con docker), en el tiempo de ejecución del filtrado de imágenes PPM y PGM en comparación con una implementación secuencial?

Los archivos de imágenes en formato PPM (Portable Pixmap) y PGM (Portable Graymap) son simples, sin comprimir e ideales para entender los conceptos básicos del procesamiento de imágenes. Además, el contenido suele ser texto plano o binario sin procesar, esto permite la fácil transferencia, lectura e incluso entendimiento del contenido y su presentación visual.

Diseño 1: Aplicación base

Desarrollar un programa en C++ que pueda leer y procesar imágenes en formato PPM (Portable Pixmap) y PGM (Portable Graymap). Utilice el paradigma orientado a objetos de tal manera que pueda extender la capacidad a medida que se consideren otros requerimientos.

El formato de los archivos PPM y PGM es el siguiente:

- Línea 1: El "número mágico", que es P3 para PPM y P2 para PGM.
- Línea 2: Los comentarios, que son opcionales y comienzan con #.
- Línea 3: El ancho y el alto de la imagen, separados por un espacio, en píxeles.
- Línea 4: El valor máximo de color (generalmente 255).

- Líneas siguientes: Los valores de los píxeles. (Si es P2, cada píxel se representa con un número (intensidad); Si es P3, cada píxel se representa con tres números)

Debe cargar archivos de color (PPM) o escala de grises (PGM) de forma automática no interactiva. La lectura se realizará desde la entrada estándar (stdin).

Shell

```
./processor lena.ppm lena2.ppm ##lectura desde entrada estándar
```

Diseño 2: Versión secuencial

Los filtros de procesamiento de imágenes son operaciones que modifican los valores de los píxeles y logran un efecto visual. Modifique la aplicación base de tal forma que permita uno o varios filtros.

- Implemente los filtros: suavizado (blur), laplace y realce (sharpening)
- Ponga a prueba su implementación con las entradas lena, fruit y puj

Shell

```
./filterer fruit.ppm fruit_blur.ppm --f blur ## filtro blur a fruit
```

- Mida y registre el tiempo en CPU y de ejecución total que le toma al computador realizar el filtrado.

Utilice como base este repositorio para comprender los filtros y su aplicación.

https://github.com/japeto/netpbm_filters

Diseño 3: Memoria compartida

Utilice las librerías threads o pthreads con C++ para implementar la paralelización de los filtros. Modifique la versión secuencial para que:

1. Se divida un archivo de entrada en cuatro regiones, arriba-izquierda, arriba-derecha, abajo-izquierda y abajo-derecha
2. Se entregue a los hilos una de las regiones de la imagen y el filtro a aplicar
3. Ponga a prueba su implementación con las entradas damma y sulfur

Shell

```
./th_filterer fruit.pgm fruit_blur2.ppm --f blur ## filtro blur a fruit
```

4. Mida y registre el tiempo en CPU y de ejecución total que le toma al computador realizar el filtrado

Utilice OpenMP para implementar la paralelización de los filtros. Modifique la versión secuencial para que:

1. Se apliquen los filtros paralelo a una imagen de entrada
2. Ponga a prueba su implementación con las entradas damma y sulfur

Shell

```
./omp_filterer sulfur.pgm sulfur_N.pgm ## filtros a sulfur
```

3. Mida y registre el tiempo en CPU y de ejecución total que le toma al computador realizar el filtrado

Diseño 4: Memoria distribuida

Implemente un esquema de paso de mensajes entre contenedores Docker con MPI.

1. Modifique la versiones secuencial y usando MPI permita que los nodos (hosts) intercambien imágenes o sus regiones y apliquen los tres filtros.
2. Ponga a prueba su implementación con al menos dos de las entradas
3. Mida y registre el tiempo en CPU y de ejecución total que le toma a cada nodo realizar el filtrado.

Recomendaciones

- Asegúrese de que su código es modular y sigue el paradigma orientado a objetos
- Utilice arreglos en lugar de vectores y punteros a Char en lugar de strings
- Revisé los enlaces y busqué información adicional sobre el formato de los archivos de entrada y sus características.

Entrega

Realice el análisis de los diseños, luego responda las siguientes preguntas.

1. ¿Por qué los diseños pueden o no aprovechar múltiples núcleos del procesador?
2. ¿Qué estructuras de datos utilizó para cada diseño? ¿Se pueden mejorar?
3. ¿Qué tan fácil es agregar nuevos filtros?
4. El tamaño de la imagen tiene impacto en la eficiencia y desempeño de los diseños

Criterios de Evaluación

Criterio	
Realiza la implementación de cada diseño	30
Presenta un análisis de ventajas y desventajas de cada diseño	20
Responde las preguntas y establece una comparación en términos de rendimiento, complejidad del desarrollo y mantenimiento posterior del sistema.	20

Justifica las decisiones de implementación de cada diseño, estructura de datos utilizadas, funciones,	10
Presenta los anteriores criterios en su informe y hace una descripción general de estos en el video	20

Notas

- Se recomienda investigar sobre los filtros sobel,

Reflexión

Un diseño paralelo no siempre necesita "*más hilos*" para ser más rápido. El ingeniero de software paralelo debe dividir un problema o tarea compleja en partes más pequeñas y manejables para ejecutarlas al mismo tiempo en distintos procesadores.

Letra pequeña

- La entrega debe realizarse en las fechas establecidas a través del aula digital.
- No se recibirán trabajos a través de correo electrónico.
- Agregue al informe los enlaces al código, al video o a cualquier otro recurso que haga parte de la entrega.
- La longitud del video debe ser menor o igual a 15 mins, todos los miembros del equipo deben participar.
- Tenga en cuenta los criterios de evaluación descritos en la Rúbrica (Si existen).
- Solo se reciben archivos en formato PDF a través de la plataforma.
- La implementación de cada Diseño debe realizarse en una rama de su repositorio